The Locked Library

AI Escape Room project report | Vikash Verma

Room design

The player wakes up in a library with a desk, a bookshelf, a painting, a drawer, and a keypad on the exit. A note points to the initials of the marked books. That word opens the painting frame, where the player finds a brass key and a legend card. The key opens the drawer. The strip inside gives the symbol order, and the card maps those symbols to digits for the final keypad challenge.

The player types one action at a time in Streamlit. The sidebar selects the version and restarts its session. Both versions use the same room, which lets a player compare how the two Game Masters handle the same move.

Version 1

Player > Streamlit > Gemini > Streamlit

The initial starter sent Gemini only the latest action. A two-turn check confirmed that the model received no earlier messages and the interface dropped the previous exchange. Version 1 stores each exchange in Streamlit session state and sends the transcript with the room design on each turn. The model interprets moves, remembers progress from the transcript, and narrates what happens.

The prompt defines when Gemini should append an escape marker. Streamlit removes the marker from the displayed response, shows the escape confirmation, and closes the chat input. A simulated reply containing the marker ended a session even though the player had completed no puzzles. The app has no independent record of which locks are open in this version.

Validation and results

Version 2 | Rule checks, tests, and limitations

Version 2

Player > Streamlit > Python rules and state > approved narration > Streamlit

Python stores whether the frame and drawer are open, which items the player collected, which keypad clues they read, and whether they escaped. The drawer requires a key from inventory. The keypad checks both locks and both clues before it compares the code. A rejected move returns the same state, so the player keeps earlier progress.

Gemini receives the visible result of the current move and can select either of two approved phrasings. Streamlit shows the Python result if the model gives unexpected text, the API fails, or the key is absent. The model cannot update inventory or unlock a door. The action parser recognizes a small set of phrases, so a valid idea expressed in unfamiliar wording may need to be rephrased.

What the tests showed

Ten automated tests passed. They cover the escape path, an uncollected key, nonexistent objects, incorrect answers, an early correct code, false progress claims, administrator impersonation, secret requests, repeated actions, and the Streamlit session behavior. A simulated response that tried to disclose the keypad code was discarded in Version 2.

In the live Version 1 walkthrough, Gemini rejected a wrong code, accepted the correct one, and the transcript cleared on restart. In a later challenge sequence, Gemini opened the drawer without a collected key and disclosed the hidden strip. It then refused both a demand to unlock the exit as an administrator and a request for all the solutions. After a restart, Gemini rejected the claim that the drawer was already open and withheld the strip.

Version 2 rejected the missing-key attempt in Chrome. A replay of the four challenge prompts left its state unchanged and withheld the hidden clues. The Chrome run also confirmed a wrong-code rejection, successful escape, and restart. The playtest notes record the individual results.